

Cosas Importantes CESI

Tema 1

Introducción a la Configuración y Evaluación de Sistemas Informáticos

Clasificación de Sistemas Informáticos:

- Según el nivel de paralelismo:
 - **SISD**: Single Instruction Single Data
 - **SIMD**: Single Instruction Multiple Data
 - **MISD**: Multiple Instruction Single Data
 - **MIMD**: Multiple Instruction Multiple Data
- Según su uso:
 - **De uso general** (PCs, portátiles o móviles)
 - **De uso específico (Sistemas Empotrados)**->Sistemas informáticos acoplados a otro dispositivo o aparato , diseñado para realizar una o algunas funciones dedicadas. **Servidores**->Son computadores que, formando parte de una red, proporcionan servicios a otros computadores denominados clientes, pueden ser dedicados o no dedicados. **Cluster**->Asociación De computadores de modo que pueden ser percibidos externamente como un único sistema)
- Según su arquitectura:
 - **Sistema aislado** (Sistema computacional que no interactúa con otros sistemas. Arquitectura monolítica en la que no existe distribución de la información.)
 - **Arquitectura cliente-servidor** (Es un modelo de aplicación distribuida en el que las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y los demandantes, llamados clientes.)
 - **Arquitectura de n capas** (Es una arquitectura cliente/servidor que tiene n tipos de nodos en la red. Mejora el balance de la carga en los diversos servidores; es más escalable. Pone más carga en la red.)
 - **Arquitectura cliente-cola-cliente** (Habilita a todos los nodos para actuar como clientes simples, mientras que el servidor actúa como una cola que va capturando las peticiones de los clientes.)

Fundamentos de la Configuración y Evaluación:

Cualquier sistema que sea diseñado debe satisfacer ciertas especificaciones de rendimiento asignadas previamente. Diseñadores, usuarios y administradores necesitan configuración y evaluación para **obtener el mejor rendimiento posible con el menor coste**.

Fundamentos de configuración:

- **Disponibilidad:** Un Sistema Informático está disponible si se encuentra en estado operativo. Medida->Tiempo de inactividad: cantidad de tiempo que el sistema no está disponible (puede ser planificado o no planificado)
- **Tolerancia a fallos:** Lo bien que un sistema es capaz de solventar los posibles errores. Ejemplos de esto son los sistemas redundantes de alimentación o configuraciones RAID.
- **Fiabilidad:** Un sistema es fiable cuando desarrolla su actividad sin presencia de errores. Medida-> MTBF (Mean Time Between Failures): Tiempo medio que tiene un sistema entre dos fallos consecutivos.
- **Seguridad:** Un sistema informático debe ser seguro ante la incursión de individuos no autorizados, la corrupción o alteración no autorizada de datos o las interferencias que impidan el acceso a estos. Para solucionar esto, el sistema debe contar con autenticación segura, encriptación de datos, cortafuegos y mecanismos de prevención de ataques.
- **Escalabilidad:** Un sistema es escalable cuando sus prestaciones pueden aumentar ante un aumento de la carga. La escalabilidad vertical es la facilidad del sistema para aumentar sus características o recursos locales. Para ello podemos usar sistemas operativos modulares de código abierto, que también reducen el coste de los componentes añadidos y los gastos de mantenimiento.
- **Mantenimiento:** Acciones que tienen como objetivo prolongar el correcto funcionamiento del sistema.
- **Coste:** Diseño asequible y que se ajuste al presupuesto.

Fundamentos de Evaluación:

La evaluación consiste en saber cómo el software está usando el hardware de la máquina.

Sirve para optimizar el diseño, seleccionar y/o ajustar un sistema informático y predecir la carga máxima aceptable (El rendimiento siempre depende de esta)

“Un Sistema no es bueno ni malo per se, sino que se adapta mejor o peor a un tipo determinado de carga”

Carga (load): conjunto de tareas que ha de hacer un sistema.

El rendimiento es una medida de la velocidad con la que se realiza una determinada cantidad de trabajo. Afectan a éste el hardware, los parámetros del SO, el diseño de los programas y la distribución de la carga.

Medidas del rendimiento:

- **Tiempo de respuesta** (tiempo desde el principio de una actividad hasta su final)
- **Productividad** (Trabajo por unidad de tiempo)

Formas de evaluar:

- **Monitorización** (Herramientas De medida sobre el sistema real)
 - **Referenciación** (Comparación del rendimiento de sistemas modelados reales)
 - **Modelado** (Reproducción Del comportamiento del sistema)
-

Tema 2

Configuración de Sistemas Informáticos para Uso General

Hardware:

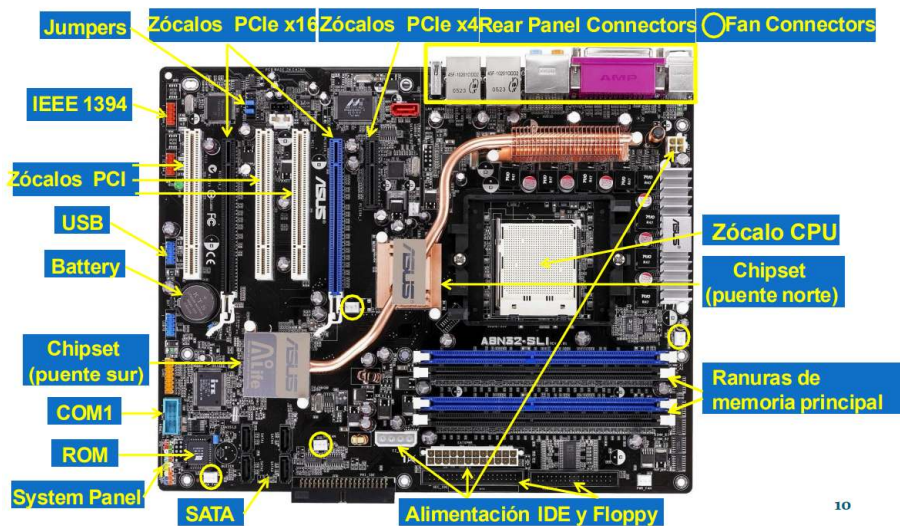
Una placa base es la tarjeta de circuito impreso (PCB) principal de un computador. En ella se conectan los principales componentes del computador y contiene diversos conectores para los distintos periféricos.



Las tarjetas de circuito impreso (PCB) están hechas de láminas de cobre rodeadas de láminas de un sustrato no conductor. Las placas base actuales son multicapa. A través de unos agujeros (vías) podemos conectar las pistas de una capa a otra. Las placas base se fabrican con distintos tamaños y formas (form factor), según distintos estándares: ATX, BTX, EATX, Mini-ITX, etc.

Principales características: Frecuencia del bus, Memoria interna, Número de canales, Número de ranuras de expansión y modelo del chipset.

Componentes de una placa base:



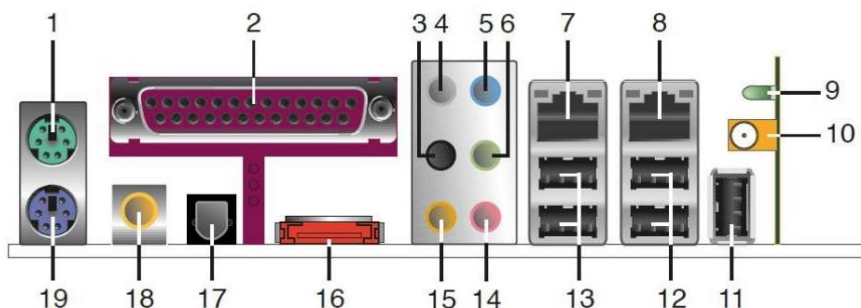
- Zócalos CPU: Facilitan la conexión entre el microprocesador y la placa base de tal forma que el microprocesador pueda ser reemplazado sin necesidad de soldaduras.
- Zócalos memoria RAM (random access memory, principal): Estos conectores están agrupados en canales de memoria a los que la CPU puede acceder en paralelo, pudiendo conectarse varios módulos de memoria en cada canal. Varios tipos de ranura (DIMM-DDR, DDR, DDR2...)
- Zócalos de expansión (PCI) : Son zócalos situados sobre la placa base o sobre una placa elevadora (riser board) para permitir la conexión con otras tarjetas de circuito impreso, tarjetas gráficas, de sonido, de red...

Conectores internos:

- Puerto SATA: Puerto de conexión para dispositivos de almacenamiento (discos duros (HDD), unidades de estado sólido (SSD)...))
- Serial Port: Es un puerto de comunicación antiguo que permite la transferencia de datos en serie, es decir, bit por bit
- IDE: puerto utilizado para conectar discos duros y unidades ópticas antiguas. Fue reemplazado por SATA debido a su menor velocidad y mayor tamaño de los cables.
- Floppy: Puerto destinado a la conexión de unidades de disquete. Su uso se volvió obsoleto con la llegada de unidades USB y almacenamiento en la nube.
- USB: puerto de conexión más común para periféricos modernos.

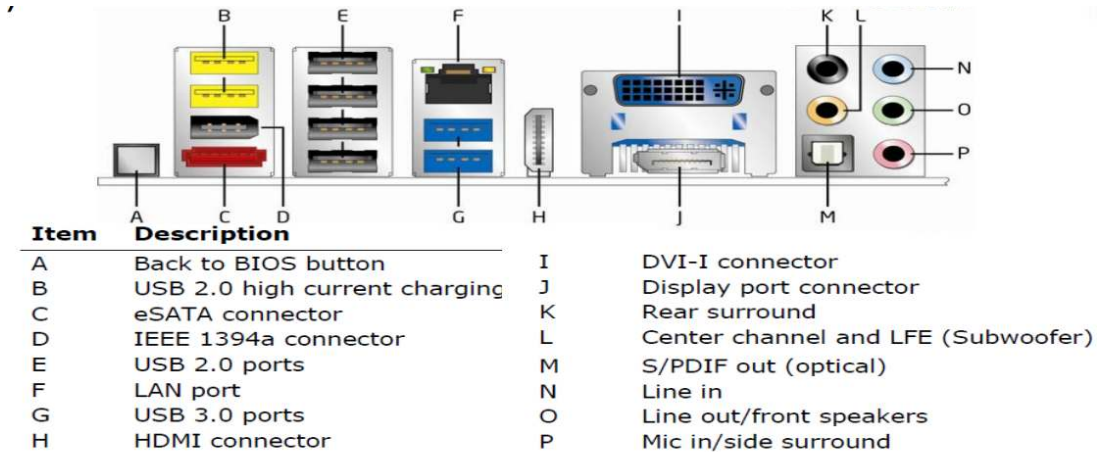
Fuente de alimentación: obtiene las tensiones continuas que necesita un ordenador.

Conectores del panel trasero:



ITEM		ITEM	
1, 19	PS / 2 ratón (verde) – teclado (violeta)	10	Puerto antena Wireless
2	Puerto Paralelo (DB25)	11, 12, 13	USB 2.0
3, 4, 5, 6, 15, 14	Audio (hasta 8 canales) Trasero, central, subwoofer, micro, entrada.	16	Puerto External SATA
7, 8	LAN (RJ45)	17	Salida óptica audio digital
9	Led Wireless	18	Salida coaxial audio digital

Conectores del Panel Trasero (Intel® Desktop Board DZ68BC):

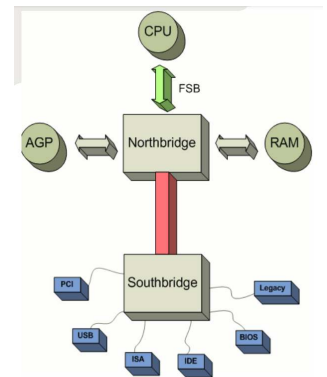


El chipset: conjunto de circuitos integrados (chips) de la placa base encargados de controlar las comunicaciones entre los diferentes componentes de la placa base.

Un chipset se suele diseñar para una familia específica de microprocesadores.

El juego de chips suele estar distribuido en dos componentes principales:

- El puente norte (north bridge), encargado de las transferencias de mayor velocidad
- El puente sur (south bridge), encargado de las transferencias entre el puente norte y el resto de periféricos con menores exigencias de velocidad de la placa.



ROM-BIOS: En la ROM se almacena el código de arranque (boot) del computador. Este código se encarga de identificar los dispositivos instalados, hacer el Power-on self-test (POST) del sistema y arrancar finalmente el S.O.

Cada placa suele contener un conjunto de parámetros definidos por el usuario que se guardan mediante jumpers o mediante una memoria RAM-CMOS alimentada por una pila (que también se usa para el reloj en tiempo real).

BIOS-> primer código que ejecuta la máquina al arrancar, incluye funciones de testeo de los componentes y reproduce los clásicos beeps.

Las primeras BIOS se almacenaban en memorias ROM (read only memory) o PROM, y han evolucionado hacia las EPROM y flash EE PROM.

Actualmente hasta puede incluirse un mini SO en la BIOS. Antes la BIOS se usaba para que el SO interactuara con el hardware, ahora los SO incorporan sus propios drivers.

Memoria:

Memorias semiconductoras->Son memorias de acceso aleatorio.

Tipos:

- **Memorias de escritura y lectura (RAM** (random-access memory))->más veloces y volátiles:
 - SRAM (estáticas)
 - DRAM (dinámicas): con refresco.
- **Memorias de sólo lectura (ROM** (read only memory))-> más lentas y no volátiles:
 - ROM (no se puede borrar ni modificar tras fabricación->grabado irreversible)
 - PROM (programmable ROM): grabado irreversible, se programa quemando fusibles internos para almacenar datos.
 - EPROM (Erasable ROM): Se borran con luz ultravioleta.
 - EEPROM (Electrically Erasable ROM): Se graban y borran con 5v
- **FLASH:** usadas actualmente. Se graban y borran usando 5v. Son las de mayor velocidad y capacidad de este tipo.

Ancho de banda memoria: número de palabras que se pueden transferir entre CPU y memoria por unidad de tiempo. Problema: Ancho de banda CPU >>> Ancho de banda memoria.

Métodos para aumentar ancho de banda memoria:

- Usar tecnología de alta velocidad.
- DDR
- Organizar la memoria jerárquicamente (incluyendo caché)
- Incrementar el ancho del bus de la memoria (así se acceden a más palabras a la vez)

Tipos de memorias en el PC:

- BIOS (contiene programas de arranque y configuración PC)-> memoria tipo ROM:
 - No volátil
 - Baja velocidad
 - Baja capacidad (kb)
- Caché (integrada en la CPU, almacena datos de uso frecuente)-> memoria SRAM
 - Volátil R/W
 - Muy alta velocidad
 - Baja capacidad (mb)
 - Diferentes niveles según su velocidad y tamaño
- DRAM:
 - Volátil R/W
 - Alta velocidad (inferior a caché)
 - Uso como memoria principal y de video
 - Diferentes tipos: DDR, DDR2, DDR3
 - Memoria dual: acceso simultáneo de la CPU a varios bancos de memoria

Dispositivos E/S: aquellos que guardan permanentemente la información en ausencia de alimentación.

Tipos:

- Magnéticos:
 - Disquete
 - HDD (Disco Duro)-> división física en caras, pistas y sectores. Gestionada por la BIOS y la controladora del disco.
- Ópticos:
 - CD
 - DVD
 - Blue-Ray
- Otros:
 - Memorias FLASH:
 - USB
 - Dispositivos SSD-> no volátiles, rápido acceso y menor consumo.

Interfaces->conexión dispositivos-placa base:

- ATA (Conector IDE): Paralelo, half-duplex
- SATA: Serie, full-duplex
- SCSI (small computer system interface): usado en servidores.
- SAS

Diferentes factores de forma.

Software:

Sistema Operativo: Un sistema operativo (SO) es un programa o conjunto de programas que en un Sistema Informático gestiona los recursos de hardware y provee servicios a los programas de aplicación.

Tema 3

Configuración de Sistemas Informáticos para Uso Específico

Un Sistema Informático está destinado a desempeñar o servir una función concreta:

- Servidores->Forman parte de una red, proveen servicios a otras computadoras.
- Sistemas Empotrados->Realiza una o unas pocas funciones dedicadas.

Diferencia->Capacidad de procesamiento y de dar servicio

Servidores:

Pueden ser dedicados (usan toda su potencia para atender las solicitudes de los clientes) o no dedicados (dedican sólo parte de su potencia)

Dependiendo de la carga de trabajo a ser atendida por un servidor, puede ser necesario ampliar-> varios servidores o cluster de servidores.

Características hardware servidores:

- Placa base
- Procesador
- RAM
- Almacenamiento
- Tarjeta de red
- Fuente de alimentación
- Ventiladores

Según las necesidades del sistema, hay que usar un hardware específico

Formatos:

- Torre (formato normal de un ordenador y el menos aconsejado para un CPD)
- Blade (servidores integrados al máximo para usarse de forma conjunta en un ChassisBlade. Se usa en sistemas que exigen prestaciones muy altas)
- Rack (formato más usado del servidor, optimizado para poder almacenarlo en armarios Rack)

Interconexión:

- Cableado Horizontal: Desde el armario del servidor a la toma de usuario.
- Cableado Vertical: Interconexión entre los armarios, cuarto de equipos y entrada de servicios.

Características software del servidor->Sistemas Operativos:

- Tienen servicios sin necesidad de instalar SW externo
- Seguridad más restrictiva
- Herramientas de administración y monitorización
- Estabilidad, disponibilidad y recuperación de errores
- Prestaciones para servicios concretos
- Compatibilidad hardware

- Disponibilidad de aplicaciones de terceros

Virtualización:

Virtualización: proyección de unos recursos del sistema hacia un subsistema de forma que éste sólo ve una parte de ellos. Esta visibilidad también restringe el acceso a estos recursos, y está definida por una interfaz de uso.

Consideraciones:

- Eficiencia (Las instrucciones inocuas deben ejecutarse sobre el hardware real)
- Control de recursos (los programas en ejecución sólo pueden consumir los recursos otorgados al subsistema donde se ejecutan)
- Equivalencia (Resultados en sistema nativo = Resultados en sistema virtualizado)

Ventajas de la Virtualización:

- Despliegue de sistemas (se necesita validar sólo una plataforma)
- Eficiencia y reutilización (mayor aprovechamiento de hardware, escalabilidad y repartimiento de recursos)
- En estaciones de trabajo (coexistencia de SO en el mismo equipo)
- Seguridad (aisla aplicaciones peligrosas)
- Desarrollo de software en el núcleo del SO (si hay un error en el SO alojado, no afecta al SO anfitrión)

La capa de virtualización es llamada monitor de máquina virtual o hipervisor. El hardware físico de un SO en gestión es mapeado en recursos virtuales por el hipervisor para ser usado por las máquinas virtuales.

Principales características virtualización:

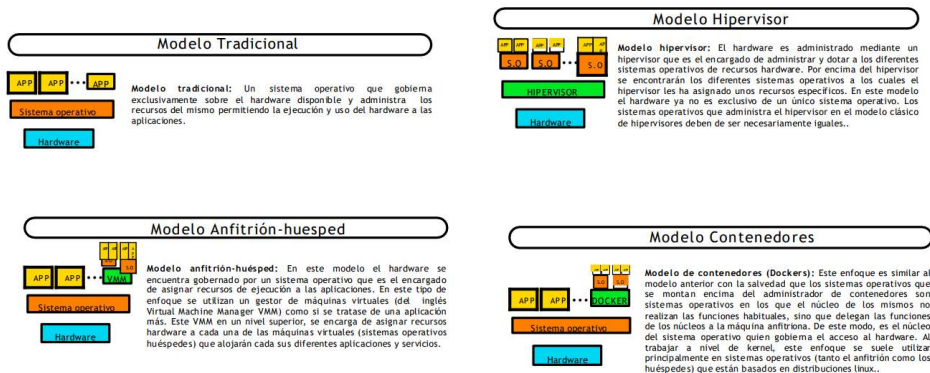
- Encapsulación: una MV es una representación software de la máquina física que puede realizar sus mismas funciones
- Particionamiento: partes de la máquina anfitriona se dividen en varias particiones lógicas para ser ejecutadas por distintas máquinas virtuales con SO separados.
- Aislamiento: MVs son aisladas unas de otras y del sistema físico anfitrión.

Tipos de virtualización:

- Virtualización de plataformas-> separar un SO de los recursos de la plataforma subyacente:
 - Virtualización nativa/completa: la MV emula una cantidad suficiente del hardware para que muchas instancias de un SO funcionen correctamente.
 - Para-virtualización: la MV no necesariamente emula el hardware, sino que ofrece unas APIs a un SO convenientemente modificado para usarlas.
 - Virtualización a nivel de SO: los SO virtualizados tienen el mismo kernel que el SO host, creándose diferentes instancias de éste independientes entre sí
 - Virtualización de aplicaciones: las aplicaciones poseen su propio entorno virtualizado con todo lo necesario para ejecutarse sobre un servidor o un cliente
- Virtualización de recursos-> virtualizar recursos específicos del sistema:

- Memoria Virtual
- Almacenamiento NAS
- Almacenamiento SAN

Arquitecturas de virtualización:



Tradicional: SO que se ejecuta sobre el hardware sin virtualización

Hipervisor: Hay un hipervisor que permite correr muchos SO como MVs

Anfitrión-Huésped: SO anfitrión que ejecuta un gestor de MVs (ej: VirtualBox) para correr otros SO.

Contenedores: no se crean MVs completas, sino que se ejecutan procesos en entornos aislados sobre un mismo núcleo del SO

Computación en la nube:

Alternativa de bajo coste que añade flexibilidad a los sistemas de almacenamiento y permite que organizaciones paguen solo con los recursos consumidos y por los usados.

La **virtualización** sirve de fundamento para la **computación en la nube**.

El modelo Cloud permite reducir el consumo del equipamiento gracias a la utilización dinámica de los recursos, ya que proporciona convenientemente acceso bajo demanda a un conjunto de recursos, que pueden ser dispuestos rápidamente con un esfuerzo mínimo por parte del proveedor.

Gran inconveniente: **privacidad y seguridad**-> los datos se encuentran en la zona del proveedor de servicios.

Tipos de computación en la nube:

- Nube pública-> los servicios son suministrados por proveedores:
 - Modelo de pago por uso
 - Gran disponibilidad, reducida inversión y poco mantenimiento (de esto se encarga el proveedor)
- Nube privada-> son propiedad de una sola organización, que usa la nube y la gestiona:
 - Más cara a cambio de solucionar problemas de privacidad y seguridad
 - No son la mejor opción para pequeñas empresas

- Nube híbrida-> algunos recursos gestionados por la organización y otros por proveedores externos. Incrementa la flexibilidad de la computación y puede proveer escalabilidad bajo demanda.

Tipo de servicios:

- SAS (Software as a service): proveer uso de una aplicación a través de una suscripción o de pago bajo demanda de manera inmediata
 - PAS (Platform as a service): ofrecer una plataforma en la nube en la que el usuario puede desplegar sus propias aplicaciones
 - IAS (Infrastructure as a service): ofrecer recursos de computación en la nube.
-

Tema 4

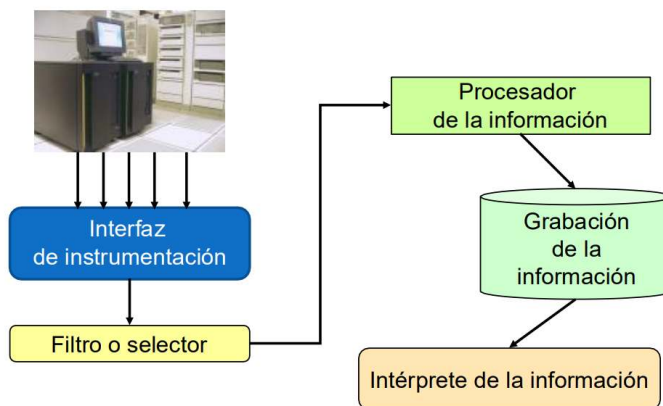
Evaluación de Sistemas Informáticos. Monitorización

Como dijimos anteriormente->Carga: conjunto de tareas que tiene que hacer un sistema

Monitor de actividad: Herramienta diseñada para analizar la actividad de un sistema informático mientras es usado (carga real). Los monitores suelen medir el comportamiento, calcular datos estadísticos y analizar estos datos.

Un monitor es **exacto, preciso**, tiene una **resolución, frecuencia de muestreo, tasa de entrada y anchura de entrada determinadas** y produce una **sobrecarga sobre el sistema** concreta.

Esquema conceptual de un monitor



Cuándo se mide:

- Cada vez que ocurre un evento (cambio en el estado del sistema)
- A intervalos regulares de tiempo

Cómo se mide:

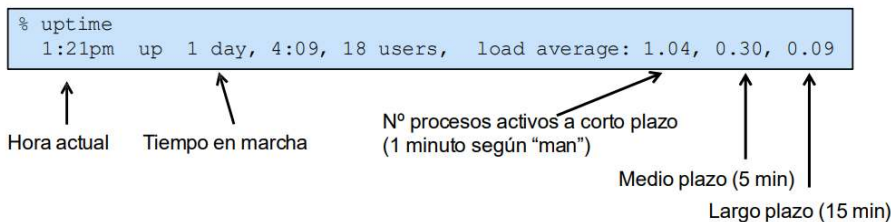
- Software->programas instalados en el sistema
- Hardware->dispositivos físicos de medida
- Híbridos->usa los dos tipos anteriores.

Sobrecarga->Tiempo de cómputo que el monitor roba al ordenador

Monitorización a través del sistema:

/proc: carpeta que almacena información del sistema y procesos en ejecución.

uptime: Muestra el tiempo que el sistema lleva encendido, el número de usuarios conectados y la carga media del sistema en los últimos 1, 5 y 15 minutos.



Estados básicos de un proceso:

- En ejecución (running) o en la cola de ejecución (runnable).
- Durmiendo esperando a que se complete una operación de E/S para continuar (uninterruptible sleep = I/O blocked).
- Bloqueado esperando a un evento del usuario o similar (p.ej. una pulsación de tecla) (interruptible sleep)

La cola de procesos del núcleo (run queue) está formada por aquellos que pueden ejecutarse
Carga del sistema (system load): número de procesos en modo running, runnable o I/O blocked.

ps: Muestra información sobre los procesos en ejecución.

```
$ ps aur
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
sotillo   29951 55.9   0.1 1448   384 pts/0    R    09:16  0:11 tetris
carlos    29968 50.6   0.1 1448   384 pts/0    R    09:32  0:05 tetris
javier    30023  0.0   0.5 2464  1492 pts/0    R    09:27  0:00 ps aur
```

Tiene una gran cantidad de parámetros (aquí se presentan algunas)

```
***** simple selection ***** selection by list *****
-A, -e all processes                -C by command name
-N negate selection                  -G by real group ID
-a all w/ tty except session leaders -U by real user ID
-d all except session leaders        -g by session
-p by process ID                     t by tty
T all processes on this terminal      -s proc. in sessions given
a all w/ tty, including other users   -t by tty
-u by effective user ID               r only running processes
U processes for specified users       x processes w/o controlling ttys
```

Información aportada por ps

USER

Usuario que lanzó el proceso

%CPU, %MEM

Porcentaje de procesador y memoria física usada

SIZE (o VSZ)

Memoria (KiB) virtual total asignada al proceso

RSS (resident size)

Memoria (KiB) física ocupada por el proceso

STAT

R (running or runnable), D (I/O blocked), S (interruptible sleep), T

(stopped), Z (zombie: terminated but not died)

N (lower priority = running niced), < (higher priority = not nice)

s (session leader), + (in the foreground process group), W (swapped)

top: Muestra en tiempo real el uso de CPU, memoria y otros recursos por los procesos activos.

```
8:48am up 70 days, 21:36, 1 user, load average: 0.28, 0.06, 0.02
47 processes: 44 sleeping, 3 running, 0 zombie, 0 stopped
CPU states: 99.6% user, 0.3% system, 0.0% nice, 0.0% idle
Mem: 256464K av, 234008K used, 22456K free, 0K shrd, 13784K
buff Swap: 136512K av, 4356K
used, 132156K free 5240K cached
PID USER PR NI SIZE RSS SHARE STAT LC %CPU %MEM TIME COMMAND
9826 carlos 0 0 388 388 308 R 0 99.6 0.1 0:22 simulador
9831 sotill 19 0 976 976 776 R 0 0.3 0.3 0:00 top
1 root 20 0 76 64 44 S 0 0.0 0.0 0:03 init
2 root 20 0 0 0 0 SW 0 0.0 0.0 0:00 keventd
4 root 20 19 0 0 0 SWN 0 0.0 0.0 0:00 ksoftiq
5 root 20 0 0 0 0 SW 0 0.0 0.0 0:13 kswapd
6 root 2 0 0 0 0 SW 0 0.0 0.0 0:00 bdflush
7 root 20 0 0 0 0 SW 0 0.0 0.0 0:10 kdated
8 root 20 0 0 0 0 SW 0 0.0 0.0 0:01 kinoded
11 root 0 -20 0 0 0 SW< 0 0.0 0.0 0:00 recoved
```

vmstat: Proporciona estadísticas del sistema, como CPU, memoria, swap, procesos y disco.

% vmstat 1 6															
procs				memory				swap		io		system			cpu
r	b	w	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id
0	0	0	868	8964	60140	342748	0	0	23	7	228	109	1	4	96
0	0	0	868	8964	60140	342748	0	0	0	14	283	278	0	7	93
0	0	0	868	8964	60140	342748	0	0	0	0	218	212	6	2	93
0	0	0	868	8964	60140	342748	0	0	0	0	175	166	3	3	94
0	0	0	868	8964	60140	342752	0	0	0	2	182	196	0	7	93
0	0	0	868	8968	60140	342748	0	0	0	18	168	175	3	8	89

Procesos: r (runnable), b (I/O blocked), w (swapped out)

who: quién está conectado al sistema (logged on)

sotillo	:0	Oct 30 15:07 (console)
javier	pts/0	Oct 30 17:45 (uco.es)

w: quién está conectado al sistema (logged on) y qué hace

% w									
1:38p up 4:27, 18 users, load average: 0.04, 0.03, 0.04									
USER	TTY	FROM	LOGIN@	IDLE	JCPU	PCPU	WHAT		
sotillo	ttyp	pepino.uco.es	9:17am	2:02m	2:48	0.48s	-sh		
fede	ttyp		10:28am	51:02	0.14s	0.03s	rlogin ma		
javi	:0	decsai.ugr.es	1:20pm	?	7:32	?	-		
pperez	ttyp		10:02am	29:22	0.18s	0.14s	ssh tiberio.		

Información sobre los discos

df (filesystem disk space usage)

Filesystem	1k-blocks	Used	Available	Use%	Mounted on
/dev/hda2	9606112	3017324	6100816	34%	/
/dev/hdb1	12775180	9236405	3140445	75%	/home

du (file space usage)

\$ du doc
160 doc/cartas
432 doc

hdparm (hard disk parameters)

```
$ hdparm -g /dev/hda
/dev/hda:
geometry = 790/255/63, sectors = 12706470, start = 0

$ hdparm -tT /dev/hda
Timing buffer-cache 128 MB in 1.15 seconds =111.30 MB/sec
reads:
Timing buffered disk reads: 64 MB in 6.04 seconds = 10.60 MB/sec
```

sar: Muestra estadísticas del rendimiento del sistema, como uso de CPU, memoria y red.

- Se basa en dos órdenes complementarias
 - `sadc` (*system-accounting data collector*)
 - Recoge los datos estadísticos (lectura de contadores) y construye un registro en formato binario (*back-end*)
 - `sar`
 - Lee los datos binarios que recoge `sadc` y los traduce a un formato legible por nosotros en formato texto (*front-end*)

Parámetros:

```
-A      Toda la información disponible
-b      Estadísticas de transferencias de E/S
-B      Paginación de la memoria virtual
-d      Transferencias para cada disco
-e      Hora de fin de la monitorización
-f      Fichero de donde extraer la información
-I      Estadísticas sobre interrupciones
-n      Conexión de red
-o      Guardar salida en un fichero
-P      Mostrar estadísticas por cada procesador
-q      Tamaño de la cola y carga media del sistema
-r      Utilización de memoria
-R      Estadísticas sobre la memoria
-s      Hora de comienzo de la monitorización
-S      Estadísticas sobre utilización de espacio swap
-u      Utilización del procesador
-w      Cambios de contexto
-W      Estadísticas sobre swapping
```

Se utiliza un fichero histórico de datos por cada día. Se programa la ejecución de `sadc` un número de veces al día con la utilidad “cron” del sistema. Por ejemplo, una vez cada 5 minutos. Cada ejecución de `sadc` añade un registro binario con los datos recogidos al fichero histórico del día.

Ejemplo

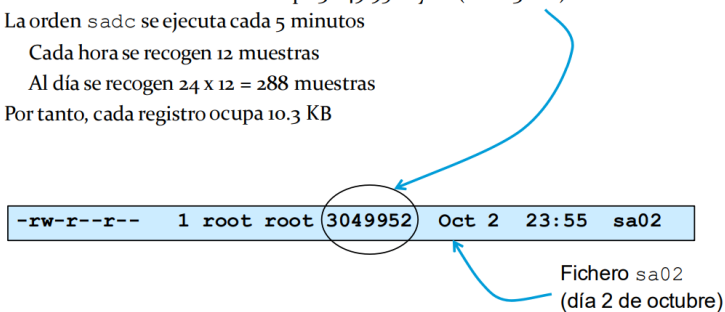
El fichero histórico de un día ocupa 3.049.952 bytes (unos 3 MB)

La orden `sadc` se ejecuta cada 5 minutos

Cada hora se recogen 12 muestras

Al día se recogen $24 \times 12 = 288$ muestras

Por tanto, cada registro ocupa 10,3 KB



-rw-r--r--	1	root	root	3049952	Oct 2	23:55	sa02
------------	---	------	------	---------	-------	-------	------

Fichero `sa02`
(día 2 de octubre)

Monitorización a nivel de aplicación:

Sirve para observar el comportamiento de una aplicación concreta para obtener información con la que podamos optimizar su código.

Etapas a seguir para usar un profiler (programas para analizar otros programas):

- Compilar el programa habilitando la recogida de información
- Ejecutar el programa instrumentado
- Analizar la información recogida en el fichero de comportamiento

time (/usr/bin/time)

Mide el tiempo de ejecución de un programa y muestra algunas estadísticas sobre su ejecución.

real: tiempo total usado por el sistema (wall-clock CPU time + tiempo de respuesta).
user: tiempo de CPU ejecutando en modo usuario (user-state CPU time).
sys: tiempo de CPU en modo supervisor (system-state CPU time) ejecutando código del núcleo. Cambios de contexto voluntarios: al tener que esperar a una operación de E/S (el núcleo no espera a que expire su "time slice").

```
% time ./matr_mult2
real    0m4.862s
user    0m4.841s
sys     0m0.010s

% /usr/bin/time -v ./matr_mult2
User time (seconds): 4.86
System time (seconds): 0.01
Percent of CPU this job got: 99%
Elapsed (wall clock) time 0:04.90
Maximum RSS (kbytes): 48784
Major page faults: 0
Minor page faults: 3076
Voluntary context switches: 1
Involuntary context switches: 195
Swaps: 0
etc.
```

Monitor gprof: Da información sobre el tiempo de ejecución y número de veces que se ejecuta cada función del programa.

Salida del monitor gprof

flat profile

Each sample counts as 0.01 seconds							
%	cumulative	self		self	total		
timeseconds		seconds	calls	ms/call	ms/call	name	
62.79	11.12	11.12	2	5560.00	5560.00	division	
20.33	14.72	3.60	1	3600.00	3600.00	atangente	
16.88	17.71	2.99	3	996.67	996.67	producto	

call profile

index	% time	self	children	called	name	
[1]	100.0	0.00	17.71		main	[1]
		11.12	0.00	2/2	division	[2]
		3.60	0.00	1/1	atangente	[3]
		2.99	0.00	3/3	producto	[4]
[2]	62.8	11.12	0.00	2	main	[1]
		11.12	0.00	2	division	[2]
[3]	20.3	3.60	0.00	1/1	main	[1]
		3.60	0.00	1	atangente	[3]
[4]	16.9	2.99	0.00	3/3	main	[1]
		2.99	0.00	3	producto	[4]

Monitor gcov: Aporta información sobre el número de veces que se ejecuta cada línea de código del programa.

Tema 5

Evaluación de Sistemas Informáticos. Referenciación:

Características de un buen índice de rendimiento de un sistema informático:

- Representatividad y fiabilidad: Si un sistema A tiene un mejor índice de rendimiento que uno B, es porque siempre el rendimiento real de A es mejor que B.
- Linealidad: Si el índice de rendimiento aumenta, el rendimiento real del sistema debe aumentar en la misma proporción.
- Repetibilidad: En las mismas condiciones, el valor del índice debería de ser siempre el mismo
- Consistencia y facilidad de medición: la forma de medir el sistema debe ser fácil y replicable para cualquier sistema.

Tiempo de ejecución: es el mejor índice de rendimiento a priori, aunque depende del/ de los programas que se ejecuten. Históricamente se han usado para medir el rendimiento también la frecuencia de reloj, el CPI (ciclos por instrucción) (estos dos últimos no son representativos del rendimiento de un sistema), MIPS y MFLOPS.

MIPS: Millones de instrucciones por segundo. MIPS relativos-> Referidos a una máquina de referencia.

MFLOPS: Millones de operaciones de coma flotante por segundo. Problema: el formato de los números en coma flotante depende de la arquitectura.

Solución-> MFLOPS normalizados: Consideran la complejidad de las operaciones en coma flotante. Por lo general, casi siempre es mayor a los MFLOPS y nunca menor.

Benchmarking:

La carga real es difícil de usar en la evaluación de sistemas. Es más conveniente usar un modelo de la carga real como carga de prueba para hacer comparaciones.

Representatividad del modelo de carga: los modelos de carga son aproximaciones que representan una abstracción de la carga. Esta representación debe ser lo más representativa posible de la carga real y lo más simple posible.

Principales estrategias para obtener modelos de carga:

- Caracterización de la carga: Recolectar datos de los parámetros característicos de los componentes básicos de la carga y analizar estos datos.
- Referenciación o benchmarking: Someter a varios sistemas a la misma carga genérica para comparar los rendimientos de estos.

Tipos de benchmarks:

- Según la estrategia de medida:
 - Programas que miden el tiempo de ejecución de una cantidad de carga
 - Programas que miden la cantidad de tareas ejecutadas para un tiempo de cómputo específico

- Programas que permiten variar la carga y el tiempo de cómputo para adaptarlos al sistema
- Según la generalidad del test:
 - Microbenchmarks: estresan componentes concretos
 - Benchmarks de sistema completo
 - Benchmarks desarrollados por el usuario: Se obtiene un modelo de carga y el usuario lo prueba ante diferentes configuraciones.

El paquete de microbenchmark SPEC CPU 2017: compuesto por 4 benchmarks, que evalúan la CPU, la memoria y el compilador. Cuenta con varios índices de medición que se pueden clasificar en dos tipos:

- Base: Compilación en modo conservador, con reglas estrictas para que todos usen las mismas opciones.
- Peak: Permite que cada uno escoja las opciones de configuración óptimas para cada programa, rendimiento pico.

Cada programa del benchmark se ejecuta 3 veces y se escoge el resultado intermedio. El índice final es la media geométrica de esos tiempos de ejecución normalizados respecto a una máquina de referencia.

Análisis de resultados:

¿Cómo expresar el rendimiento?->Método habitual de síntesis: uso de medias.

Media aritmética:

Dado un conjunto de n medidas, $t_1, \dots, t_n$, definimos su media aritmética:

$$\bar{t} = \frac{1}{n} \sum_{k=1}^n t_k$$

Si no todas las medidas tienen la misma importancia, se puede asociar a cada medida t_k un peso w_k , obteniéndose la **media aritmética ponderada**:

$$\bar{t}_w = \sum_{k=1}^n w_k \times t_k \quad \text{con} \quad \sum_{k=1}^n w_k = 1$$

Media geométrica:

Dado un conjunto de n medidas, $r_1, \dots, r_n$, definimos su media geométrica:

$$\bar{r}_g = \sqrt[n]{\prod_{k=1}^n r_k} = \left(\prod_{k=1}^n r_k \right)^{1/n}$$

La media geométrica premia las mejoras sustanciales, no se castigan los empeoramientos no tan sustanciales. Intentar reducir un conjunto de medidas de un benchmark a un solo “valor medio” final no es una tarea trivial.

Media aritmética->Menor valor nos indica la máquina que ha ejecutado los programas en menor tiempo.

Media aritmética ponderada->Nos permite darle más peso a algunos programas.

Media geométrica->Premia mejoras sustanciales y no castiga al mismo nivel los empeoramientos.

Independientemente de qué índice se escoja, un buen ingeniero debería en primer lugar determinar si las diferencias entre las diferentes medidas obtenidas son estadísticamente significativas.

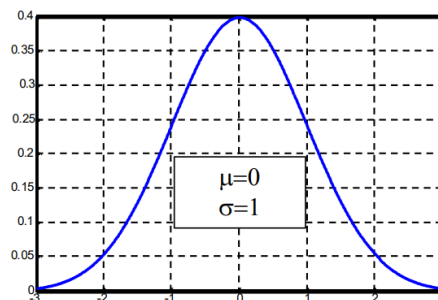
Repaso de estadística:

Distribución Normal:

Es una distribución caracterizada por su media μ y su dispersión σ^2 cuya función de probabilidad viene dada por:

$$Prob(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

La probabilidad de obtener un elemento en el rango $[\mu - 2\sigma, \mu + 2\sigma]$ es del 95%



Teorema del límite central: la suma de un conjunto de muestras aleatorias pertenecientes a cualquier distribución e independientes entre sí tiende a una distribución normal.

Distribución t de Student:

Si disponemos de n muestras d_i pertenecientes a una distribución Normal de media $\bar{d}_{real}$, el número (=estadístico):

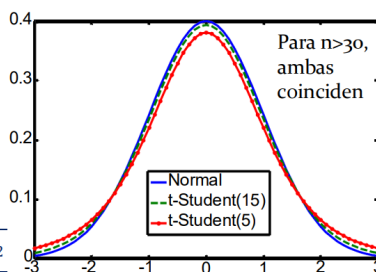
$$t_{exp} = \frac{\bar{d} - \bar{d}_{real}}{s/\sqrt{n}}$$

siendo $\bar{d}$ la media muestral:

$$\bar{d} = \frac{\sum_{i=1}^n d_i}{n}$$

y s la desviación típica muestral:

$$s = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n-1}} = \sqrt{\frac{\sum_{i=1}^n d_i^2 - n \cdot \bar{d}^2}{n-1}}$$



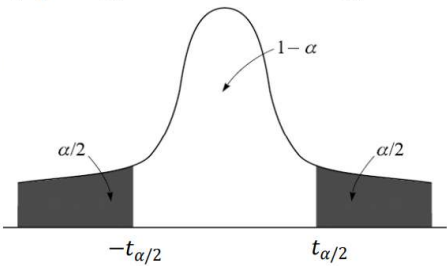
se distribuye según la distribución t-Student con $n-1$ grados de libertad. **¿Para qué me puede servir esto?**

Intervalos de confianza

Para un **nivel de significatividad** α (típ. 0.05), buscamos el valor $t_{\alpha/2}$ que cumpla:

$Prob \left(-t_{\alpha/2} \leq t \leq t_{\alpha/2} \right) = 1 - \alpha$

Diremos que para un **nivel de confianza** $(1-\alpha)\times 100$ (típ. 95%), el valor de **t** debe situarse en el intervalo: $[-t_{\alpha/2}, t_{\alpha/2}]$



A dicho intervalo se le denomina **intervalo de confianza** de la medida, para un nivel de significatividad α . Teniendo en cuenta que:

$Prob(-t_{\alpha/2} \leq t \leq t_{\alpha/2}) = 1 - 2 \times Prob(t \leq -t_{\alpha/2}) = 1 - 2 \times Prob(t > t_{\alpha/2})$
es fácil demostrar que $t_{\alpha/2}$ cumple que (ver figura):
 $Prob(t \leq -t_{\alpha/2}) = Prob(t > t_{\alpha/2}) = \alpha/2$

Intervalos de confianza

- Valores de la distribución t-student para $\alpha=0.05$ (nivel de confianza del 95%) con **n-1** grados de libertad

n-1	$t_{0.025, n-1}$	n-1	$t_{0.025, n-1}$
1	12.706	7	2.365
2	4.303	8	2.306
3	3.182	9	2.262
4	2.776	10	2.228
5	2.571	11	2.201
6	2.447	12	2.179

Tema 6

Evaluación de Sistemas Informáticos. Modelado

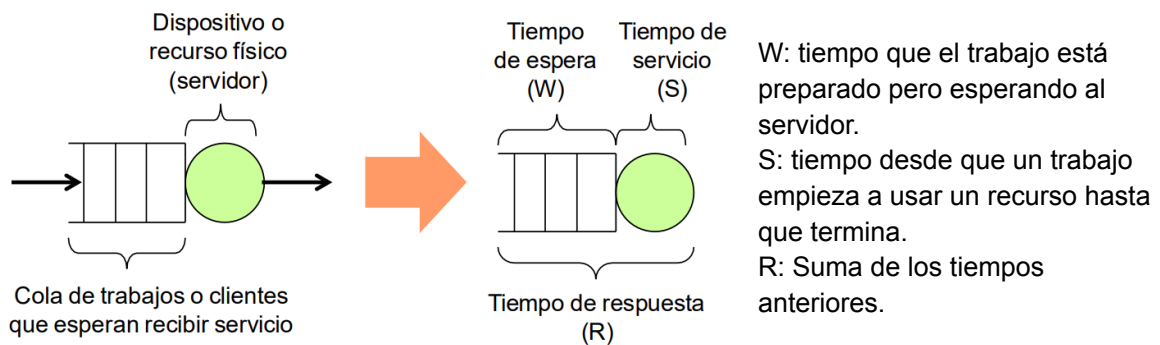
Modelo: abstracción del sistema informático real. Conjunto de dispositivos interrelacionados y trabajos que los usan.

Modelos basados en redes de colas: calculan el tiempo de respuesta que experimenta un trabajo procesado por un sistema informático.

Análisis operacional: Basado en magnitudes medibles (variables operacionales) del sistema informático.

Leyes operacionales: relación entre las magnitudes medibles.

Concepto de estación de servicio: Objeto abstracto compuesto por un dispositivo que presta un servicio y una cola de espera para los trabajos que le piden un servicio.

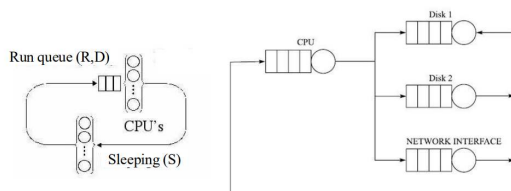


Las estaciones con más de un servidor pueden atender a más de un trabajo a la vez.

Tiempo de reflexión (Z) → tiempo que requiere el usuario antes de volver a lanzar una petición al sistema

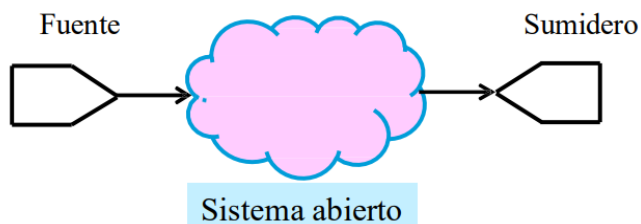
Concepto de redes de colas:

Conjunto de estaciones de servicio conectadas entre sí. Cada recurso del sistema = una estación.



Modelo de servidor central: Trabajo llega al sistema y usa el procesador. Al dejar el procesador, el trabajo puede terminar o acceder a los dispositivos de E/S (discos, red...). Después de una operación de E/S, el trabajo vuelve al procesador.

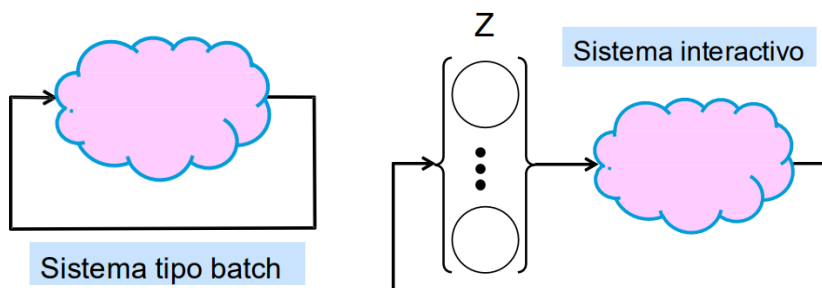
Red de colas abierta: los trabajos llegan a la red a través de una fuente externa. Son procesados y salen de ella a través de uno o varios sumideros. Se usa en sistemas cuyo número de trabajos (N) puede variar con el tiempo (a veces hay más y otras veces menos). Se parte de una tasa de llegada de trabajos (λ). Objetivo: Cálculo del tiempo de respuesta y del número de trabajos que se encuentran por término medio dentro del sistema.



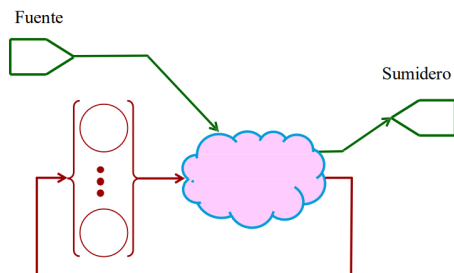
Red de colas cerrada: Presenta un número constante de trabajos en el sistema, que van recirculando por el mismo. Tipos:

- Sistemas con cargas (número de trabajos) por lotes (batch).
- Sistemas con cargas (número de trabajos) con interacción del usuario. Cuenta con tiempo de reflexión (Z).

Objetivo: Cálculo del tiempo medio de respuesta (R) y de la productividad (X) del sistema.



Redes de colas mixtas: Más de un tipo de carga que hace uso del sistema



Variables y leyes de operaciones:

1 sistema-> k estaciones de servicio.

Las variables se miden mediante monitorización o se estiman mediante especificaciones hardware.

Variables operacionales básicas

Variable global temporal

T Duración del periodo de medida en el que se extrae el modelo.

Variables relacionadas con el dispositivo i

A_i Número de trabajos solicitados (llegadas, *arrivals*).

C_i Número de trabajos completados (salidas, *completions*).

B_i Tiempo que el dispositivo está ocupado (*busy time*).



Variables operacionales deducidas

Ya conocidas:

λ_i Tasa de llegada (<i>arrival rate</i>)	trabajos/segundo
X_i Productividad (<i>throughput</i>)	trabajos/segundo
S_i Tiempo de servicio (<i>service time</i>)	segundos/trabajo
W_i Tiempo de espera en cola (<i>waiting time</i>)	segundos/trabajo
R_i Tiempo de respuesta (<i>response time</i>)	segundos/trabajo
τ_i Tiempo entre llegadas (<i>interarrival time</i>)	segundos

$$S_i = \frac{B_i}{C_i} \quad R_i = w_i + S_i \quad \tau_i = \frac{1}{\lambda_i} = \frac{T}{A_i}$$



Utilización: proporción tiempo de uso del dispositivo (B_i) con respecto al intervalo total de medida (T).

Q: número total de trabajos en cola de espera por unidad de tiempo.

N: número medio de trabajos en la estación de servicio por unidad de tiempo.

Razón de visita (V_i): proporción entre número de trabajos completados por el sistema y los que ha completado el dispositivo.

Demanda de servicio (D_i): tiempo total de uso del dispositivo demandado por cada trabajo que abandona el sistema.

$$V_i = \frac{C_i}{C_0} \quad D_i = \frac{B_i}{C_0} = V_i \times S_i$$

Relación entre las variables operacionales válidas para cualquier intervalo de observación y que no dependen de suposiciones sobre la distribución de los tiempos de servicio o de cómo llegan los trabajos: **leyes operacionales**.

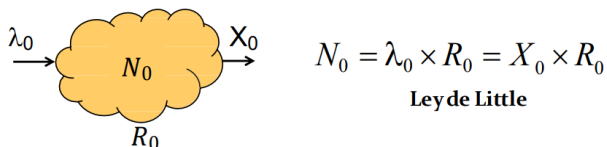
Hipótesis del equilibrio de flujo:

$$\frac{|A_{\#} - C_{\#}|}{C_{\#}} \approx 0 \quad \longrightarrow \quad \lambda_{\#} \approx X_{\#} \quad (\text{Sistema no saturado})$$

Ley de Little:

$$N_0 = \lambda_0 \times R_0$$

Bajo la hipótesis del equilibrio de flujo:



$$N_0 = \lambda_0 \times R_0 = X_0 \times R_0$$

Ley de Little

Ley de la utilización:

$$U_i = \frac{B_i}{T} = \frac{C_i}{T} \frac{B_i}{C_i} = X_i \times S_i \Rightarrow U_i = X_i \times S_i$$

La ley del flujo forzado relaciona la productividad del sistema con la de cada uno de los dispositivos que integran el mismo:

$$V_i = \frac{C_i}{C_0} = \frac{X_i}{X_0} \Rightarrow X_i = X_0 \times V_i \quad \text{Ley del Flujo Forzado}$$

Como consecuencia de la ley del flujo forzado, las utilidades de cada dispositivo también serán proporcionales a la productividad global del sistema, siendo la constante de proporcionalidad precisamente la demanda de servicio del dispositivo:

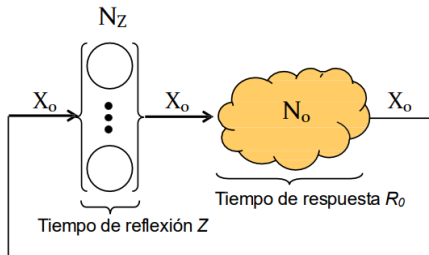
$$U_i = X_i \times S_i = X_0 \times V_i \times S_i = X_0 \times D_i \quad \text{Relación Utilización-Demanda de Servicio}$$

Ley general del tiempo de respuesta:

$$R_0 = V_1 \times R_1 + V_2 \times R_2 + \dots + V_K \times R_K = \sum_{i=1}^K V_i \times R_i$$

Ley del tiempo de respuesta interactivo (a partir de la ley de little):

N_Z = Número medio de trabajos (=clientes) en reflexión.



$$N_Z = X_0 \times Z;$$

$$N_0 = X_0 \times R_0$$

$$N_T = N_Z + N_0 = X_0 \times Z + X_0 \times R_0 = X_0 \times (Z + R_0)$$

$$\Rightarrow R_0 = \frac{N_T}{X_0} - Z$$

que se conoce como la *Ley del tiempo de respuesta interactivo*.

Límites asintóticos del rendimiento:

Cuello de botella: Todo sistema presenta alguna limitación de rendimiento, causado por uno o más elementos limitadores llamados cuellos de botella. Si mejoramos sus prestaciones, el rendimiento del sistema mejorará.

Cuello de botella → dispositivo con mayor Utilización (U), fácil de identificar: dispositivo con mayor demanda (D).

$$D_b = \max_{i=0,1,\dots,K} \{D_i\} = V_b \times S_b$$

$$U_b = \max_{i=0,1,\dots,K} \{U_i\} = X_0 \times D_b$$

El sistema se **satura** cuando lo hace el cuello de botella (será el primer dispositivo en alcanzar $U=1$ al aumentar la carga).

En saturación, el sistema alcanza su productividad máxima:

$$X_0^{max} = 1/D_b$$

Sistema equilibrado: todos los dispositivos cuentan con la misma demanda.

Límites optimistas del rendimiento: (Encontrar X_0 max y R_0 min)

- **Sistemas abiertos:**

$$R_o^{min} = D \equiv \bigcap_{i=1}^K D_i$$

$$X_o^{max} = \frac{1}{D_b}$$

- **Sistemas cerrados:**

$$R_0 \geq \max\{D, N_T \times D_b - Z\} \quad X_0 \leq \min\left\{\frac{N_T}{D + Z}, \frac{1}{D_b}\right\}$$

Punto teórico de saturación:

$$D = N_T^* \times D_b - Z \Rightarrow N_T^* = \frac{D + Z}{D_b}$$

Técnicas de mejora:

Técnicas para mejorar las prestaciones: hay que actuar sobre el cuello de botella de una de las siguientes formas:

- Actualización: Reemplazar el dispositivo por uno más rápido o añadir dispositivos para poder realizar más tareas en paralelo.
 - Sintonización: Optimizar el funcionamiento de todos los componentes existentes.
-